

Patrones Creacionales y Seguridad

Proyecto: Gestión de Eventos y Donaciones - Fundación Cudeca

Equipo Turing

1. Introducción

El presente informe detalla la evolución técnica del proyecto. El objetivo ha sido triple: 1) Desacoplar la lógica de negocio mediante Patrones de Diseño. 2) Asegurar la integridad y privacidad mediante protocolos de Seguridad. 3) Garantizar una integración visual y funcional perfecta ("Seamless Integration") con el ecosistema digital de la Fundación Cudeca.

2. Arquitectura de Software (Patrones)

Reestructuración del Backend para soportar mantenibilidad y escalabilidad.

A. Singleton

- **Herramienta / Estrategia:** AppConfig (en patterns.py)
- **Justificación:** Gestión centralizada de la configuración del entorno y rutas de archivos estáticos.
- **Problema / Riesgo:** La dispersión de rutas "hardcoded" provocaba errores al desplegar en diferentes entornos y duplicidad en la creación de directorios.
- **Solución Implementada:** Instancia única que valida la infraestructura de carpetas al inicio y provee un punto de acceso global a la configuración.

B. Factory Method

- **Herramienta / Estrategia:** PaymentFactory (Creador)
- **Justificación:** Abstracción de la lógica de pasarelas de pago (Escalabilidad horizontal).
- **Problema / Riesgo:** El acoplamiento fuerte en los controladores dificultaba añadir nuevos métodos de pago sin refactorizar todo el código.
- **Solución Implementada:** Delegación de la creación del procesador. El sistema es agnóstico a la pasarela (Bizum, Stripe, PayPal), facilitando futuras integraciones bancarias reales.

C. Builder

- **Herramienta / Estrategia:** EventoBuilder (en patterns.py)
- **Justificación:** Construcción controlada de objetos complejos con múltiples dependencias (Imágenes, Fechas, Datos).
- **Problema / Riesgo:** Controladores masivos ("Fat Controllers") que mezclaban validación, gestión de archivos y persistencia.
- **Solución Implementada:** Separación de la construcción y la representación. El Builder gestiona la complejidad de los archivos adjuntos y la validación de datos paso a paso.

D. Abstract Factory

- **Herramienta / Estrategia:** TransactionFactory
- **Justificación:** Tratamiento polimórfico de transacciones heterogéneas (Eventos vs Rifas).
- **Problema / Riesgo:** Inconsistencia en la generación de entregables: Los eventos requieren QRs, las rifas solo registros digitales.
- **Solución Implementada:** Interfaz unificada que garantiza la creación coherente de familias de objetos relacionados (Entrada+QR o Boleto+Registro) sin exponer la lógica al cliente.

E. Prototype

- **Herramienta / Estrategia:** Evento.clone() / Rifa.clone()
- **Justificación:** Optimización operativa para la gestión de eventos recurrentes.
- **Problema / Riesgo:** Pérdida de tiempo y errores humanos al recrear manualmente eventos idénticos (ej: Gala Benéfica anual).
- **Solución Implementada:** Clonación profunda de objetos en memoria para su posterior persistencia como nuevas entidades.

F. Facade

- **Herramienta / Estrategia:** FundraisingFacade (en services.py)
- **Justificación:** Proveer una interfaz unificada y simplificada para un conjunto de interfaces en un subsistema complejo (Base de datos, Ficheros, Factories).
- **Problema / Riesgo:** Los controladores de la aplicación (app.py) comenzaban a violar el Principio de Responsabilidad Única, manejando directamente sesiones de base de datos, lógica de archivos y llamadas a múltiples fábricas. Esto genera un "código espagueti" difícil de testear y propenso a errores.
- **Solución Implementada:** Se ha encapsulado toda la lógica de negocio (Donaciones, Gestión de Eventos, Compras y Metas) tras una fachada única. El

controlador web ahora solo invoca métodos de alto nivel como `process_donation` o `purchase_ticket`, desconociendo la complejidad interna de las transacciones SQL o la generación de QRs.

G. Observer

- **Herramienta / Estrategia:** Subject / EmailNotificationObserver (en `patterns.py`)
- **Justificación:** Desacoplamiento de la lógica transaccional (ej: cobrar una entrada) de la lógica reactiva (ej: enviar un email de confirmación).
- **Problema / Riesgo:** Integrar el envío de correos electrónicos directamente dentro de los métodos de pago ralentiza la respuesta al usuario y crea una dependencia fuerte. Si el servicio de email falla, podría revertir una transacción financiera válida.
- **Solución Implementada:** Implementación de un sistema de publicación/suscripción. La clase `FundraisingFacade` actúa como Subject (Sujeto). Cuando ocurre un evento importante (ej: `DONATION_RECEIVED`), el sistema "notifica" a los observadores suscritos. Esto permite añadir nuevas reacciones en el futuro (SMS, Logs de auditoría) sin tocar el código de procesamiento de pagos.

3. Estrategia de Diseño e Integración UI/UX

Dado que el propósito final es la implantación real, la interfaz se ha diseñado bajo el principio de "Integración Transparente". El usuario no debe percibir una ruptura entre la web institucional y esta plataforma de gestión.

A. Fidelidad a la Identidad Corporativa

- **Herramienta / Estrategia:** Sistema de Diseño (Design System) & CSS Variables
- **Justificación:** La marca Cudeca tiene una identidad visual consolidada que transmite esperanza y profesionalidad.
- **Problema / Riesgo:** El uso de colores genéricos o estilos predeterminados de Bootstrap generaría desconfianza y desconexión con la marca madre.
- **Solución Implementada:** Se ha implementado una hoja de estilos personalizada que replica exactamente la paleta cromática corporativa (Verde Cudeca #4CAF50 para acciones principales, Dorado para destacados) y las tipografías oficiales, asegurando que el módulo se sienta nativo.

B. Continuidad Estructural y de Navegación

- **Herramienta / Estrategia:** Base Template & Layout Consistency
- **Justificación:** La estructura de navegación (Header, Footer, Menús) debe ser inmutable durante la transición entre plataformas.
- **Problema / Riesgo:** Un cambio en la ubicación del menú o el pie de página provoca desorientación en el usuario, aumentando la tasa de rebote.
- **Solución Implementada:** Se ha diseñado un "base.html" que clona la estructura de la web principal, incluyendo logotipos, enlaces a redes sociales y avisos legales en las mismas posiciones exactas, garantizando una curva de aprendizaje nula para el usuario habitual.

C. Psicología de la Confianza en Pagos

- **Herramienta / Estrategia:** Diseño de Interfaz Orientado a Conversión (CRO)
- **Justificación:** En un entorno de donaciones, la percepción de seguridad visual es tan importante como la seguridad técnica.
- **Problema / Riesgo:** Diseños descuidados, alineaciones incorrectas o interfaces "amateur" hacen que el usuario dude de la legitimidad del formulario de pago.
- **Solución Implementada:** Se ha cuidado al píxel la maquetación de los formularios de pago (Tarjetas, Bizum), utilizando espaciados claros, feedback visual inmediato y elementos de autoridad visual para maximizar la confianza del donante.

D. Coherencia Semántica y Tono de Voz

- **Herramienta / Estrategia:** UX Writing & Microcopy
- **Justificación:** La fundación mantiene un tono empático, agradecido y cercano, alejado de la frialdad transaccional.
- **Problema / Riesgo:** El uso de términos agresivos o puramente técnicos ("Pagar", "Transacción", "Error 404") rompe la narrativa solidaria.
- **Solución Implementada:** Se ha adaptado todo el "microcopy" (textos de botones, mensajes de éxito, correos) para reflejar el vocabulario de la organización (ej: "Colaborar" en lugar de "Pagar", mensajes de agradecimiento personalizados tras la donación), manteniendo la calidez humana en el proceso digital.

4. Seguridad e Infraestructura Técnica

Implementación de capas de seguridad para la protección de datos y hardware.

A. Hashing y Protección de Credenciales

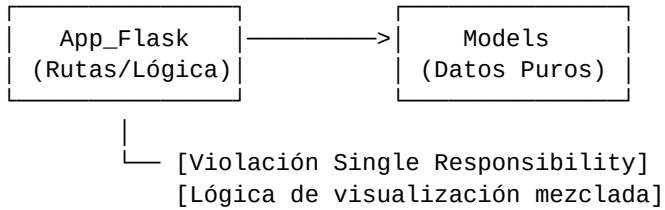
- **Herramienta / Estrategia:** Werkzeug Security (PBKDF2-SHA256)
- **Justificación:** Protección crítica de la identidad digital de socios y administradores.
- **Problema / Riesgo:** El almacenamiento de contraseñas en texto plano es inaceptable bajo normativa GDPR y estándares de seguridad (OWASP).
- **Solución Implementada:** Implementación de hashing unidireccional con "salt". La base de datos nunca conoce la contraseña real, haciendo inútil la información en caso de filtración.

B. Protocolo Seguro HTTPS (SSL/TLS)

- **Herramienta / Estrategia:** Flask SSL Context (Ad-hoc Certificate)
- **Justificación:** Requisito mandatorio para el acceso a periféricos del cliente (Cámara para escaneo QR).
- **Problema / Riesgo:** Los navegadores modernos (Chrome/Safari) bloquean la API getUserMedia en contextos no seguros (HTTP), inutilizando el escáner de entradas.
- **Solución Implementada:** Despliegue bajo HTTPS utilizando certificados autofirmados para desarrollo y preparación para certificados Let's Encrypt en producción, habilitando el uso seguro de la cámara y el cifrado de datos en tránsito.

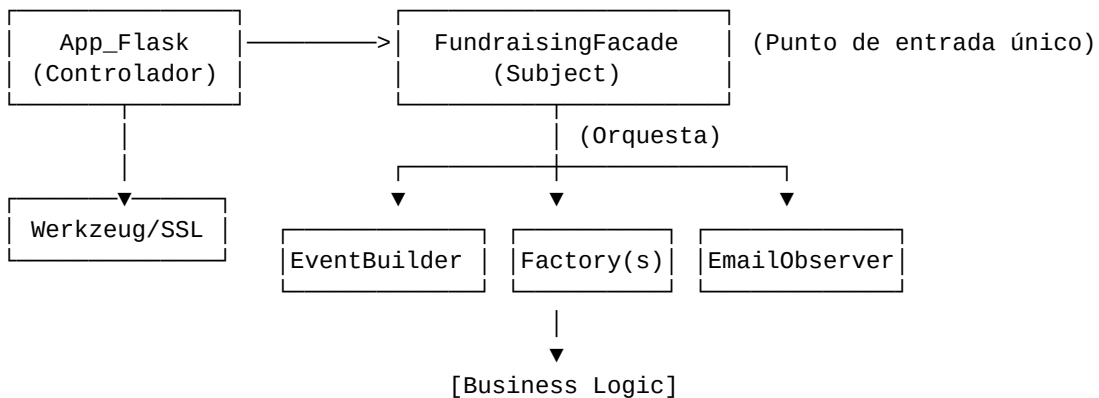
5. Diagramas de Arquitectura

Fase 1: Enfoque Monolítico (Inicial)



Fase 2: Arquitectura Distribuida (Final)

Integración de Patrones, Seguridad y Capa de Presentación con Acceso Unificado.



6. Conclusión

El desarrollo realizado trasciende la simple implementación funcional. Se ha construido una solución de software profesional que equilibra una arquitectura interna robusta (Patrones y Seguridad) con una capa de presentación cuidadosamente diseñada para integrarse en la realidad operativa de la Fundación Cudeca.

El resultado es un sistema escalable, seguro y, sobre todo, empático con el usuario final.